

Dry run Sieve algorithm

Absolutely. I'll explain it in the same teaching style—starting from the problem, building the intuition, then deriving the algorithm naturally instead of jumping directly to the code.

Step 1: Understand the Problem

For every query n , we need to count **how many prime numbers $\leq n$** can be written as

$$x^2 + y^4$$

where

- x is any positive integer
- y is any positive integer

and the resulting number must itself be **prime**.

Example 1

Suppose

```
n = 10
```

Let's generate numbers.

Try $y = 1$

Since

$$1^4 = 1$$

Now vary x .

x	x^2	x^2+1	Prime?
1	1	2	✓
2	4	5	✓
3	9	10	✗

We found

```
2
5
```

Try $y = 2$

$$2^4 = 16$$

Smallest possible value

$$1^2 + 16 = 17$$

Already greater than 10.

So we stop.

Answer

```
2
```

Example 2

```
n=20
```

Generate numbers.

$y=1$

```
1^2+1=2
2^2+1=5
```

```
32+1=10
42+1=17
```

Prime numbers

```
2
5
17
```

y=2

```
24=16
12+16=17
22+16=20
```

17 is already counted.

No new prime.

Answer

```
2
5
17
Count = 3
```

Notice something important.

The same number may be generated multiple times.

Example

```
17
=12+24
=42+14
```

We only count it once.

That is why we use

```
good[value]=true;
```

instead of increasing some counter.

Step 2: Think About Constraints

Maximum

```
n = 10,000,000
```

Queries

```
T = 10,000
```

Imagine doing this for every query.

For each query

```
try every x
try every y
check prime
```

That would be extremely slow.

So instead we preprocess everything **once**.

Step 3: How many x values exist?

We know

```
x2+y4 ≤ 107
```

Since

```
y4 ≥ 1
```

then definitely

$$x^2 \leq 10^7$$

Take square root.

$$x \leq \sqrt{10^7}$$

Calculate

$$\sqrt{10000000} \approx 3162$$

So

x goes only from
1
to
3162

Not millions.

Only

3162 values.

Step 4: How many y values exist?

Similarly

$$y^4 \leq 10^7$$

Take fourth root.

$$y \leq \sqrt[4]{10^7}$$

Calculate

$$56^4 = 9834496$$

$$57^4 = 10556001$$

Already exceeds limit.

Therefore

y
only goes from
1
to
56

Only

56 values!

Step 5: Total possibilities

Now total pairs

$$3162 \times 56 \approx 177072$$

That's tiny.

Instead of billions of possibilities, we only check

177 thousand.

This is the key observation.

Step 6: How do we know whether $x^2 + y^4$ is prime?

Checking primality repeatedly is expensive.

Instead we first compute all primes.

Use

Sieve of Eratosthenes

After sieve

```
isPrime[17]=true
isPrime[18]=false
isPrime[19]=true
...
```

Now checking primality becomes

```
if(isPrime[value])
```

which is

```
O(1)
```

Step 7: Sieve

Initially

Assume everything is prime.

```
Index
0 1 2 3 4 5 6 7 8 9 10
T T T T T T T T T T
```

We know

```
0
and
1
are not prime.
```

```
F F T T T T T T T T
```

Start from

```
2
```

Since

```
2 is prime
```

remove all multiples.

```
4
6
8
10
12
...
```

Next

```
3
```

Remove

```
6
9
12
15
...
```

Next

4

Already removed.

Skip.

Continue until

$\sqrt{\text{MAX}}$

because larger factors are already handled.

Finally

```
isPrime[2]=true
isPrime[3]=true
isPrime[4]=false
...
isPrime[999983]=true
```

Everything is ready.

Step 8: Generate x^2+y^4

Now

```
for every y
for every x
```

Compute

```
value= $x^2+y^4$ 
```

Example

```
y=1
y4=1
```

x=1

```
12+1
=2
```

Prime?

```
Yes
good[2]=true
```

x=2

```
4+1
=5
```

Prime?

```
Yes
good[5]=true
```

x=3

```
9+1
=10
```

Prime?

```
No
```

Nothing happens.

Eventually

```
x2+y4
becomes
10000050
```

Already exceeds MAX.

Stop this loop.

Next

```
y=2
16
```

Again

```
12+16
17
Prime?
Yes
```

```
good[17]=true
```

Eventually

```
y=57
574
>
10000000
```

Stop.

Finished.

Step 9: What does good[] store?

Suppose we found

```
2
5
17
37
```

Then

```
good[2]=true
good[5]=true
good[17]=true
good[37]=true
```

Everything else

```
false
```

Visual

```
0 F
1 F
2 T
3 F
4 F
5 T
6 F
```

```
...
17 T
```

Step 10: Why Prefix Sum?

Imagine

```
10000 queries
```

If every query scans from

```
1
to
n
```

Time

```
10000 * 100000000
```

Too much.

Instead

Create prefix.

Suppose

```
good
0 1 2 3 4 5 6 ...17
F F T F F T F ...T
```

Build

```
prefix
```

Start

```
prefix[0]=0
```

i=1

```
good[1]=false
prefix[1]=0
```

i=2

```
good[2]=true
prefix[2]
=
prefix[1]+1
=
1
```

i=3

```
prefix[3]
=
1
```

i=4

```
1
```

i=5

```
good[5]=true
prefix[5]=2
```

Eventually

```
Index
0 1 2 3 4 5 6 ...
Prefix
0 0 1 1 1 2 2 ...
```

Meaning

```
How many valid numbers
≤i
```

Now query

```
n=10
```

Answer

```
prefix[10]
```

Instantly

```
2
```

Query

```
9999999
```

Answer

```
prefix[9999999]
```

Again

```
O(1)
```

Step 11: Understanding the Code

Step 1

```
C++
vector<bool> isPrime(MAX+1, true);
```

Initially

```
Every number
assumed prime.
```

Step 2

```
C++
vector<bool> good(MAX+1, false);
```

Initially

```
No number has yet been found in
x²+y²
```

Step 3

```
C++
vector<int> prefix(MAX+1, 0);
```

Stores

```
Answer for every possible n.
```


Sieve

```
C++
isPrime[0]=false;
isPrime[1]=false;
```

Because

```
0
and
1
are not prime.
```

Loop

```
C++
for(long long i=2;i*i<=MAX;i++)
```

Only till

```
√MAX
```

because every composite number has at least one factor less than or equal to its square root.

If

```
C++
isPrime[i]
```

then remove multiples.

```
C++
for(long long j=i*i;j<=MAX;j+=i)
```

Notice

Starts from

```
i²
```

not

```
2i
```

because smaller multiples have already been removed by smaller prime numbers.

Example

For

```
5
```

```
10
```

```
15
```

```
20
```

already removed by

```
2
```

```
3
```

```
4
```

Only

```
25
```

needs to start.

Generating y^4

```
C++
long long y4=y*y;
y4*=y4;
```

Instead of writing

```
pow(y,4)
```

which uses floating-point arithmetic and is slower.

Example

```
y=5
25
25×25
625
```

Exactly

```
54
```

Breaking the Loop

```
C++
if(y4>MAX)
break;
```

Because larger values of y will only make y^4 even bigger, so there is no point continuing.

Inside

```
C++
value=x*x+y4;
```

If

```
value>MAX
```

break.

Because

```
x
only increases.
x2
only increases.
```

No future value can become smaller again.

Prime Check

```
C++
if(isPrime(value))
```

Constant time.

No division.

No loops.

Already precomputed.

Mark

```
C++
good[value]=true;
```

Even if

```
17
```

appears twice

it remains

true

No duplicate counting.

Prefix

```
C++
prefix[i]=prefix[i-1];
```

Copy previous answer.

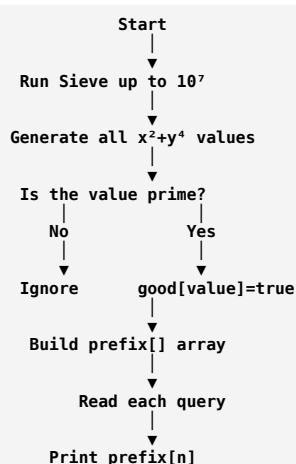
If current number is valid

```
C++
prefix[i]++;
```

Now

```
prefix[i]
=
number of valid primes
≤i
```

Overall Flow



Time Complexity

1. Sieve

$$O(MAX \log \log MAX)$$

where $MAX = 10^7$.

2. Generate $x^2 + y^4$

- x ranges up to about **3162**
- y ranges up to **56**

Total iterations:

$$3162 \times 56 \approx 177,000$$

This is negligible compared to the sieve.

3. Prefix Array

$$O(MAX)$$

4. Each Query

Just:

```
C++
cout << prefix[n];
```

which is:

$$O(1)$$

Final Complexity

- **Preprocessing:** $O(MAX \log \log MAX)$
- **Per query:** $O(1)$

The entire solution is efficient because we do the expensive work only once, then answer every query instantly using the prefix array. This is a classic **precomputation + sieve + prefix sum** problem.

Let's derive the **time complexity of the Sieve of Eratosthenes** mathematically, step by step.

Code

```
C++
for (int i = 2; i * i <= n; i++) {
    if (isPrime[i]) {
        for (int j = i * i; j <= n; j += i) {
            isPrime[j] = false;
        }
    }
}
```

There are **two loops**, so we'll analyze each one.

Step 1: Outer Loop

```
C++
for (int i = 2; i * i <= n; i++)
```

Condition is

```
C++
i * i <= n
```

Taking square root,

```
i <= √n
```

Therefore the outer loop runs

```
√n times
```

So,

$$T_{outer} = O(\sqrt{n})$$

But this **is not** the final complexity because the inner loop does different amounts of work for each **i**.

Step 2: Inner Loop

Suppose

```
C++
i = 2
```

Then

```
C++
for (j = 4; j <= n; j += 2)
```

The values are

```
4
6
8
10
...
n
```

Number of iterations is approximately

$$\frac{n}{2}$$

Now suppose

```
C++
i = 3
```

Loop is

```
C++
j = 9, 12, 15, ...
```

Iterations

$$\frac{n}{3}$$

For

```
C++
i = 5
```

Iterations

$$\frac{n}{5}$$

For

```
C++
i = 7
```

Iterations

$$\frac{n}{7}$$

Generally,

for prime p ,

iterations are approximately

$$\frac{n}{p}$$

Step 3: Add the Work

Ignoring constants,

Total work becomes

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \dots$$

Factor out n ,

$$n \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots \right)$$

Now the question becomes

What is

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots$$

where the sum is over **primes**?

Step 4: Prime Harmonic Series

A famous result in number theory says

$$\sum_{p \leq n} \frac{1}{p} = \log \log n + O(1)$$

This means

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \dots = \log \log n$$

approximately.

Step 5: Multiply Back

Therefore

$$T(n) = n \cdot \log \log n$$

Hence

$$O(n \log \log n)$$

Why not $O(n\sqrt{n})$?

A common mistake is saying:

- Outer loop runs $\sqrt{n}$ times.
- Inner loop runs n/i times.
- Therefore,

$$\sqrt{n} \times n = n\sqrt{n}$$

This is **incorrect** because the inner loop **does not always run n times**. It runs:

i	Iterations
2	$n/2$
3	$n/3$
5	$n/5$
7	$n/7$
11	$n/11$
13	$n/13$

The number of iterations keeps decreasing as **i** increases, so you **sum** the costs instead of multiplying them.

Example for $n = 100$

The primes up to $\sqrt{100} = 10$ are:

2, 3, 5, 7

Work done:

Prime	Iterations
2	$100/2 = 50$
3	$100/3 \approx 33$
5	$100/5 = 20$
7	$100/7 \approx 14$

Total

```
50 + 33 + 20 + 14 = 117
```

This matches

$$100 \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} \right) \approx 117$$

Notice it's **much smaller** than

```
100 * sqrt(100) = 1000
```

which is why the complexity is **not** $O(n\sqrt{n})$.

Intuition

The sieve is fast because:

1. **Only prime numbers** perform marking. Composite numbers are skipped due to `if (isPrime[i])`.
2. **Each prime marks only its multiples**, which is about n/p operations.
3. The sum of the reciprocals of the primes grows extremely slowly, like $\log \log n$, leading to the final complexity:

$\text{Time Complexity} = O(n \log \log n)$

and the space complexity is:

$O(n)$

This is a very good question. The confusion is:

If the outer loop runs only $\sqrt{n}$ times, why does the final complexity become $O(n \log \log n)$ instead of $O(\sqrt{n} \log \log n)$?

The answer is **because the dominant work is done by the inner loop**, not the outer loop.

Let's understand it carefully.

Step 1: Outer loop

```
C++
for(int i = 2; i * i <= n; i++)
```

The number of iterations is

$$\sqrt{n}$$

For example, if

```
n = 100
```

then

```
i = 2, 3, 4, 5, 6, 7, 8, 9, 10
```

Only **9 iterations**.

So the outer loop itself costs

$$O(\sqrt{n})$$

Step 2: Inner loop

Suppose

```
n = 100
```

When

```
i = 2
```

the inner loop runs

```
4
6
8
...
100
```

≈ 50 iterations

When

```
i = 3
```

it runs

```
9
12
15
...
99
```

≈ 33 iterations

When

```
i = 5
```

≈ 20 iterations

When

```
i = 7
```

≈ 14 iterations

Total work is

```
50 + 33 + 20 + 14
```

Notice something important:

The **inner loop alone has already done 117 operations**, while the outer loop only executed **9 times**.

The outer loop is almost negligible compared to the work done inside it.

Step 3: Write the sum

The total work is

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \dots$$

Factor out n :

$$n \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots \right)$$

The remaining sum is

$$\log \log n$$

So

$$T(n) = n \log \log n$$

Why don't we multiply by $\sqrt{n}$?

Many beginners think:

```
Outer loop = √n
Inner loop = n/i
Answer = √n × n
```


This is **incorrect**.

Why?

Because the inner loop **doesn't run n/i for every number from 2 to $\sqrt{n}$** in the same amount. Each iteration has a **different cost**.

Think of it like this:

i	Inner loop work
2	50
3	33
5	20
7	14

You **add** these costs:

50 + 33 + 20 + 14

You do **not** multiply:

4 × 50

because each outer-loop iteration performs a different amount of work.

General Rule for Nested Loops

If the inner loop always runs the same number of times:

```
C++
for (i=1; i<=n; i++)
    for (j=1; j<=n; j++)
```

then

$$O(n) \times O(n) = O(n^2)$$

because every outer iteration does exactly n inner iterations.

But if the inner loop depends on i :

```
C++
for (i=1; i<=n; i++)
    for (j=i; j<=n; j++)
```

the work is

$$(n) + (n - 1) + (n - 2) + \dots + 1$$

which is a **sum**, not a multiplication.

Similarly, in the sieve:

```
C++
for (i=2; i*i<=n; i++)
```

the work is

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \dots$$

which simplifies to

$$O(n \log \log n)$$

Key takeaway

- The **outer loop** only determines **how many different inner loops** are executed ($\sqrt{n}$ of them).
- The **total running time** is the **sum of the work done by all those inner loops**, not $\sqrt{n}$ multiplied by the maximum inner-loop cost.
- Since the sum of the inner-loop work is $n \log \log n$, that term dominates the $\sqrt{n}$ cost of the outer loop.

So the final complexity is:

$$O(n \log \log n)$$

not

$$O(\sqrt{n} \cdot n).$$

ChatGPT can make mistakes. Check important info.